JINK CSP-J 普及级模拟赛（一）题解

总体定位

本套题总分 400 分，比赛时间 3.5 小时，按 CSP-J 第二轮的四题形式组织。

题目	核心算法	满分复杂度	预期定位
书架整理	位置数组、贪心	O(n)	T1，基础思维
山谷补给	排序、分组贪心	O(nlog n)	T2，基础综合
遗迹通道	BFS、分层状态	O(nmk)	T3，搜索题
营地联络	树形 DP、树上背包	O(nk)，最坏 $O(n^2)$	T4，普及组压轴

第一题 书架整理

一次操作把某一本书移动到最右端，没有被移动的书之间的相对顺序始终不变。

设最终有一些书没有被移动。因为移动过的书最后都位于所有未移动书的右侧，而目标排列是 1,2,...,n，所以未移动的书只能是编号 1,2,...,t，并且它们在原排列中的位置必须严格递增。

记 `pos[x]` 为编号 x 在原排列中的位置。我们从 1 开始寻找最大的 t，满足

$$pos_1 < pos_2 < \cdots < pos_t.$$

这样可以保留前 t 本书不动，再依次把编号 t+1,t+2,...,n 的书移动到最右端，恰好使用 n-t 次操作得到目标排列。

另一方面，任何方案都不能保留比 t 更多的书，否则 `pos[t+1]` 也应位于 `pos[t]` 之后，与 t 的最大性矛盾。 因此答案为 n-t。

时间复杂度 O(n)，空间复杂度 O(n)。

第二题 山谷补给

正坐标与负坐标不能在同一趟行程中补给，因此可以先分别计算两侧所有必须返回的行程长度。

考虑公路的一侧，把所有观测点到仓库的距离从大到小排序为

$$d_1 \ge d_2 \ge \cdots \ge d_s.$$

最远的观测点 d_1 所在行程最多还能顺路补给后面的 k-1 个点，因此把 d_1,d_2,...,d_k 放入同一趟一定不劣。接下来最远的未补给点是 d_k+1，同理继续分组。

如果所有行程都需要返回，这一侧的最小代价为

$$2(d_1 + d_{k+1} + d_{2k+1} + \cdots).$$

分别对左右两侧计算并相加。最后一趟行程可以不返回；若它的最远距离为 d，就可以节省 d。显然应让最后一趟前往全体观测点中离仓库最远的位置。所以最终答案等于全部行程都往返时的最小代价， 再减去 max_i|x_i|。

时间复杂度 O(nlog n)，空间复杂度 O(n)。答案需要使用 64 位整数。

第三题 遗迹通道

只记录当前位置不能完整描述搜索状态：到达同一格时，已经使用的破障装置数量不同，后续可行性也不同。

把一个搜索状态表示为 (x,y,z)，其中 (x,y) 是当前位置，z 是已经使用的破障装置数量。若移动到普通格， z 不变；若移动到障碍格，新的状态为 (x',y',z+1)，并要求 $z+1 \le k$。

所有移动的步数代价都是 1，因此从 (S_x,S_y,0) 开始做 BFS。用三维数组 `dis[x][y][z]` 记录每个状态第一次到达时的距离。BFS 按距离从小到大扩展，所以第一次访问某个状态时得到的就是 该状态的最短距离。

最终答案为

$$\min_{0 \le z \le k} dis[T_x][T_y][z].$$

若所有这些状态均未被访问，则输出 -1。

状态数不超过 nm(k+1)，每个状态只有四条转移。时间复杂度 O(nmk)，空间复杂度 O(nmk)。

第四题 营地联络

把树根定为总部 1。一个合法方案包含某个结点 u 的子树中的结点时，如果还要与 u 的父亲方向连通，那么在 这部分被选结点中必须包含 u。

定义

$$f_{u,j} =$$

在 u 的子树中选择 j 个结点、选择的结点连通且必须包含 u 时的最大价值总和。

初始只有结点 u 自身：

$$f_{u,1} = a_u.$$

设当前正在把儿子 v 合并进 u。若原来已经从处理过的部分选择了 i 个结点，又从 v 的子树中选择了 j 个结点，那么两部分通过边 (u,v) 连通，可以转移：

$$f_{u,i+j} = \max(f_{u,i+j}, f_{u,i} + f_{v,j}).$$

也可以不从 v 的子树中选择任何结点，此时原状态保持不变。合并时让 i 从大到小枚举，可以避免同一个儿子的 状态被重复使用。

所有儿子合并完成后，`f[u]` 就包含了 u 的整棵子树。最终答案为 f_1,k。

合并时必须把枚举范围限制在已经处理的结点数与儿子子树大小内。整棵树上所有合并的枚举次数总计为 O(nk)；由于 $k \le n$，最坏时间复杂度为 $O(n^2)$。空间复杂度为 O(nk)。因为价值可能为负，所有不可达状态必须初始化为 负无穷，不能初始化为 0。

难度与预期得分说明

T1 需要把“移动到末尾”转化为保留相对顺序；T2 重点是从最远点开始分组，并正确处理最后一次不返程；T3 是本套题的 搜索题，关键在于把剩余破障能力加入 BFS 状态；T4 要发现连通选择在根树上的结构，再完成树上背包合并。

附录：标准程序

第 1 题 书架整理（shelf）标程

```
#include<bits/stdc++.h>
#define int long long
#define endl "\n"
using namespace std;
const int N=2e5+10;
int n,a[N],pos[N];
signed main()
{
    freopen("shelf.in","r",stdin);freopen("shelf.out","w",stdout);
    ios::sync_with_stdio(false);cin.tie(nullptr);
    cin>>n;
    for(int i=1;i<=n;i++)cin>>a[i],pos[a[i]]=i;
    int x=1;
    while(x<n&&pos[x]<pos[x+1])x++;
    cout<<n-x<<endl;
    return 0;
}
```

第 2 题 山谷补给（supply）标程

```
#include<bits/stdc++.h>
#define int long long
#define endl "\n"
using namespace std;
const int N=2e5+10;
int n,k;
vector<int> a,b;
int calc(vector<int> &a)
{
    sort(a.begin(),a.end(),greater<int>());int res=0;
    for(int i=0;i<a.size();i+=k)res+=a[i]*2;
    return res;
}
signed main()
{
    freopen("supply.in","r",stdin);freopen("supply.out","w",stdout);
    ios::sync_with_stdio(false);cin.tie(nullptr);
    cin>>n>>k;
    for(int i=1,x;i<=n;i++)
    {
        cin>>x;
        if(x>0)a.push_back(x);
        else b.push_back(-x);
    }
    int ans=calc(a)+calc(b),mx=0;
    if(a.size())mx=max(mx,a[0]);
    if(b.size())mx=max(mx,b[0]);
    cout<<ans-mx<<endl;
    return 0;
}
```

第 3 题 遗迹通道（maze）标程

```
#include<bits/stdc++.h>
#define int long long
#define endl "\n"
using namespace std;
const int N=510,K=12,INF=1e18;
int n,m,k,dis[N][N][K];
char a[N][N];
struct node
{
    int x,y,z;
};
signed main()
{
    freopen("maze.in","r",stdin);freopen("maze.out","w",stdout);
    ios::sync_with_stdio(false);cin.tie(nullptr);
    cin>>n>>m>>k;
    int sx=0,sy=0,tx=0,ty=0;
    for(int i=1;i<=n;i++)
        for(int j=1;j<=m;j++)
        {
            cin>>a[i][j];
            if(a[i][j]=='S')sx=i,sy=j;
            if(a[i][j]=='T')tx=i,ty=j;
        }
    memset(dis,-1,sizeof(dis));queue<node> q;
    dis[sx][sy][0]=0;q.push((node){sx,sy,0});
    int dx[4]={1,-1,0,0},dy[4]={0,0,1,-1};
    while(q.size())
    {
        node u=q.front();q.pop();
        for(int i=0;i<4;i++)
        {
            int x=u.x+dx[i],y=u.y+dy[i];
            if(x<1||x>n||y<1||y>m)continue;
            int z=u.z+(a[x][y]=='#');
            if(z>k||dis[x][y][z]!=-1)continue;
            dis[x][y][z]=dis[u.x][u.y][u.z]+1;
            q.push((node){x,y,z});
        }
    }
    int ans=INF;
    for(int i=0;i<=k;i++)if(dis[tx][ty][i]!=-1)ans=min(ans,dis[tx][ty][i]);
    cout<<(ans==INF?-1:ans)<<endl;
    return 0;
}
```

第 4 题 营地联络（camp）标程

```
#include<bits/stdc++.h>
#define int long long
#define endl "\n"
using namespace std;
const int N=3010,INF=4e18;
int n,k,a[N],f[N][N],siz[N];
vector<int> e[N];
void dfs(int x,int fa)
{
    siz[x]=1;f[x][1]=a[x];
    for(int y:e[x])
    {
        if(y==fa)continue;dfs(y,x);
        for(int i=min(siz[x],k);i>=1;i--)
            for(int j=1;j<=min(siz[y],k-i);j++)
                if(f[x][i]>-INF&&f[y][j]>-INF)
                    f[x][i+j]=max(f[x][i+j],f[x][i]+f[y][j]);
        siz[x]+=siz[y];
    }
}
signed main()
{
    freopen("camp.in","r",stdin);freopen("camp.out","w",stdout);
    ios::sync_with_stdio(false);cin.tie(nullptr);
    cin>>n>>k;
    for(int i=1;i<=n;i++)cin>>a[i];
    for(int i=1,x,y;i<n;i++)cin>>x>>y,e[x].push_back(y),e[y].push_back(x);
    for(int i=1;i<=n;i++)for(int j=0;j<=k;j++)f[i][j]=-INF;
    dfs(1,0);cout<<f[1][k]<<endl;
    return 0;
}
```